Lecture 6 – Function (Part 1)

DITG 1113 COMPUTER PROGRAMMING

LEARNING OUTCOMES

- At the end of this lecture, you should be able to
  - explain the importance of using function
  - define and call the function
  - pass data into a function
  - differentiate function that returns a value and function that does not return a value

MODULAR PROGRAMMING

- Modular programming: breaking a program up into smaller, manageable functions or modules

- Function: a collection of statements to perform a task
- Motivation for modular programming:
  - Improves maintainability of programs
  - Simplifies the process of writing programs

This program has one long, complex function containing all of the statements necessary to solve a problem.

```
int main()
{
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
   statement;
}
```

In this program the problem has been divided into smaller problems, each of which is handled by a separate function.

```
int main()
{
   statement;
   statement;
   statement;
}
```

main function

```
void function2()
{
   statement;
   statement;
   statement;
}
```

function 2

```
void function3()
{
   statement;
   statement;
   statement;
}
```

function 3

```
void function4()
{
   statement;
   statement;
   statement;
}
```

function 4

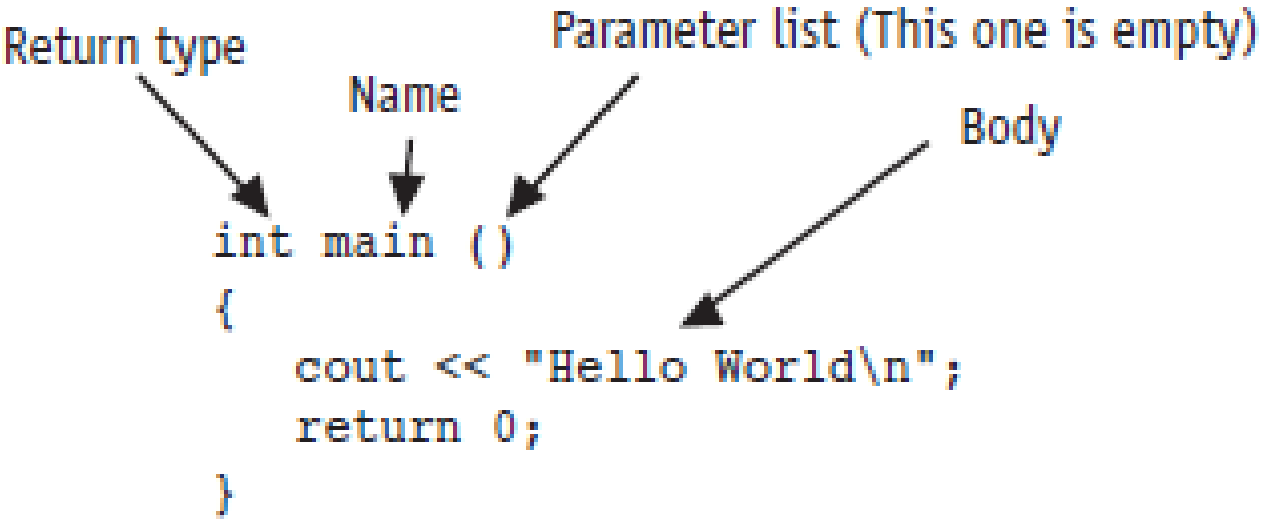
DEFINING AND CALLING FUNCTIONS

- Function call: statement causes a function to execute
- Function definition: statements that make up a function

FUNCTION DEFINITION

- Definition includes:
  - name: name of the function. Function names follow same rules as variables
  - parameter list: variables containing values passed to the function
  - body: statements that perform the function's task, enclosed in { }
  - return type: data type of the value that function returns to the part of the program that called it

FUNCTION DEFINITION



Note: The line that reads `int main()` is the *function header*.

FUNCTION HEADER

- The function header consists of
  - the function *return type*
  - the function *name*
  - the function *parameter list*
- Example:

```
int main()
```

- Note: no `;` at the end of the header

FUNCTION RETURN TYPE

- If a function returns a value, the type of the value must be indicated:

```
int main()
```

- If a function does not return a value, its return type is `void`:

```
void printHeading()
 {
   cout << "Monthly Sales\n";
 }
```

CALLING A FUNCTION

- To call a function, use the function name followed by `()` and `;`

```
printHeading();
```

- When called, program executes the body of the called function
- After the function terminates, execution resumes in the calling function at point of call.

CALLING A FUNCTION

- **`main`** is automatically called when the program starts
- **`main`** can call any number of functions
- Functions can call other functions

FUNCTION PROTOTYPES

The compiler must know the following about a function before it is called

- name
- return type
- number of parameters
- data type of each parameter

FUNCTION PROTOTYPES

Ways to notify the compiler about a function before a call to the function:

- Place function definition before calling function's definition
- Use a function prototype (similar to the heading of the function
  - Heading: `void printHeading()`
  - Prototype: `void printHeading();`

PROTOTYPE

- Place prototypes near top of program
- Program must include either prototype or full function definition before any call to the function, otherwise a compiler error occurs
- When using prototypes, function definitions can be placed in any order in the source file. Traditionally, **`main`** is placed first.

EXAMPLE

```
1  // This program has three functions: main, First, and Second.
2  #include <iostream>
3  using namespace std;
4
5  // Function Prototypes
6  void first();
7  void second();
8
9  int main()
10 {
11    cout << "I am starting in function main.\n";
12    first();     // Call function first
13    second();    // Call function second
14    cout << "Back in function main again.\n";
15    return 0;
16 }
17
```

EXAMPLE (CONTINUE)

```
18  //****************************************
19  // Definition of function first.        *
20  // This function displays a message.    *
21  //****************************************
22
23  void first()
24  {
25     cout << "I am now inside the function first.\n";
26  }
27
28  //****************************************
29  // Definition of function second.       *
30  // This function displays a message.    *
31  //****************************************
32
33  void second()
34  {
35     cout << "I am now inside the function second.\n";
36  }
```

SENDING DATA INTO A FUNCTION

- Can pass values into a function at time of call

```
c = sqrt(a*a + b*b);
```

- Values passed to function are arguments
- Variables in function that hold values passed as arguments are parameters
- Alternate names:
  - argument: actual argument, actual parameter
  - parameter: formal argument, formal parameter

PARAMETERS, PROTOTYPES, AND FUNCTION HEADINGS

- For each function argument,
  - the prototype must include the data type of each parameter in its `()`

```
void evenOrOdd(int);      //prototype
```

  - the heading must include a declaration, with variable type and name, for each parameter in its `()`

```
void evenOrOdd(int num)   //heading
```

- The function call for the above function would look like this: `evenOrOdd(val);  //call`

FUNCTION CALL

- Value of argument is copied into parameter when the function is called
- Function can have > 1 parameter
- There must be a data type listed in the prototype **()** and an argument declaration in the function heading **()** for each parameter
- Arguments will be promoted/demoted as necessary to match parameters

CALLING FUNCTIONS WITH MULTIPLE ARGUMENTS

When calling a function with multiple arguments

- the number of arguments in the call must match the function prototype and definition
- the first argument will be copied into the first parameter, the second argument into the second parameter, etc.

CALLING FUNCTIONS WITH MULTIPLE ARGUMENTS ILLUSTRATION

```
displayData(height, weight);   // call


void displayData(int h, int w)// heading
{
    cout << "Height = " << h << endl;
    cout << "Weight = " << w << endl;
}
```

PASSING DATA BY VALUE

- Pass by value: when argument is passed to a function, a copy of its value is placed in the parameter
- Function cannot access the original argument
- Changes to the parameter in the function do not affect the value of the argument in the calling function

PASSING DATA TO PARAMETERS BY VALUE

- Example:

```
int val = 5;
evenOrOdd(val); //call
void evenOrOdd(int num) //header
```

- **evenOrOdd** can change variable **num**, but it will have no effect on variable **val**

THE `RETURN` STATEMENT

- Used to end execution of a function
- Can be placed anywhere in a function
  - Any statements that follow the **`return`** statement will not be executed
- Can be used to prevent abnormal termination of program
- Without a **`return`** statement, the function ends at its last `}`

RETURNING A VALUE FROM A FUNCTION

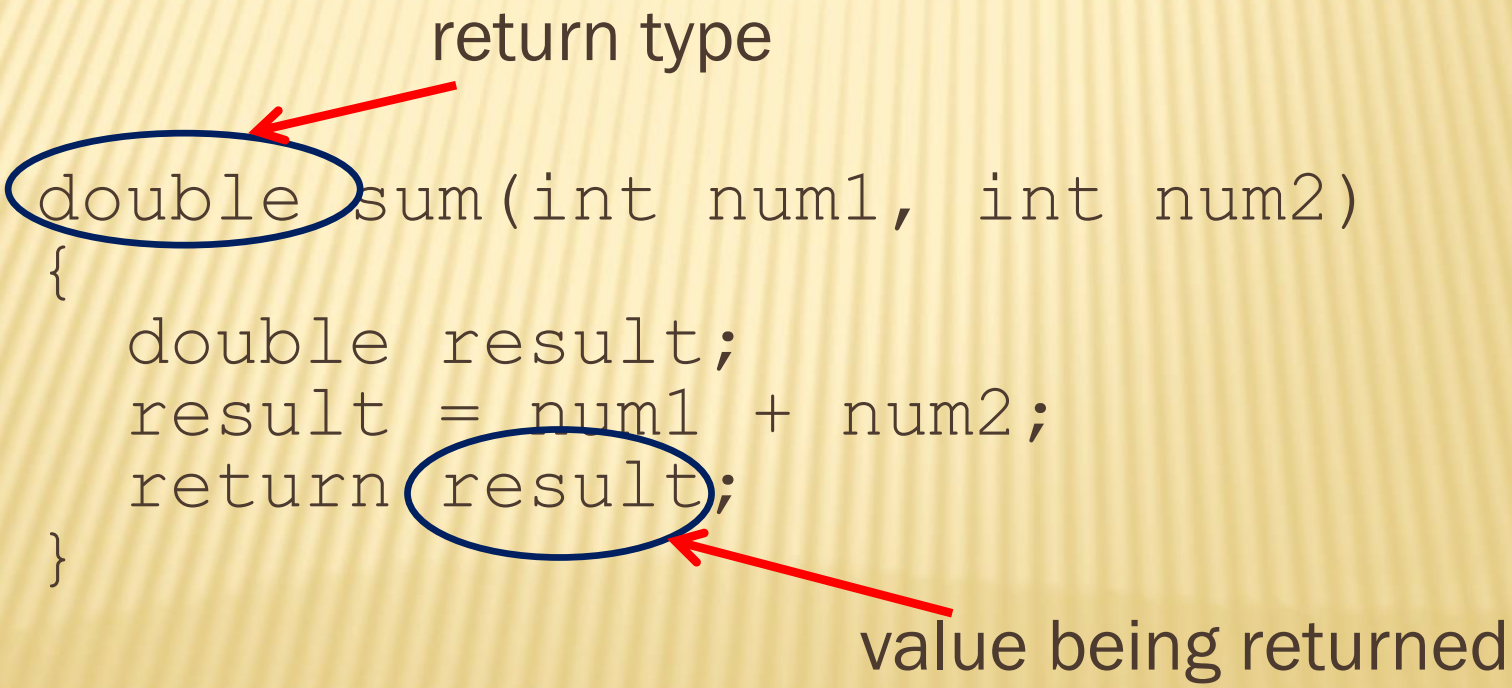
- **`return`** statement can be used to return a value from the function to the module that made the function call
- Prototype and definition must indicate data type of return value (not **`void`**)
- Calling function should use return value, e.g.,
  - assign it to a variable
  - send it to **`cout`**
  - use it in an arithmetic computation
  - use it in a relational expression

RETURNING A VALUE – THE `RETURN` STATEMENT

- Format: `return expression;`
- ***expression*** may be a variable, a literal value, or an expression.
- ***expression*** should be of the same data type as the declared return type of the function (will be converted if not)

EXAMPLE RETURNING A VALUE FROM A FUNCTION

- In a value-returning function, the `return` statement can be used to return a value from the function to the point of call. Example:



RETURNING A BOOLEAN VALUE

- Function can return **`true`** or **`false`**
- Declare return type in function prototype and heading as **`bool`**
- Function body must contain **`return`** statement(s) that return **`true`** or **`false`**
- Calling function can use return value in a relational expression

BOOLEAN RETURN EXAMPLE

```
bool isValid(int);              // prototype

bool isValid(int val)           // heading
{
    int min = 0, max = 100;
    if (val >= min && val <= max)
       return true;
    else
       return false;
}

if (isValid(score))             // call
    …
```

USING FUNCTIONS IN A MENU-DRIVEN PROGRAM

Functions can be used

- to implement user choices from menu
- to implement general-purpose tasks
  - Higher-level functions can call general-purpose functions
  - This minimizes the total number of functions and speeds program development time

LOCAL AND GLOBAL VARIABLES

- local variable: defined within a function or block; accessible only within the function or block
- Other functions and blocks can define variables with the same name.
- When the function begins, its local variables and its parameter variables are created in memory, and when the function ends, the local variables and parameter variables are destroyed.
- This means that any value stored in a local variable is lost between calls to the function in which the variable is declared.

LOCAL AND GLOBAL VARIABLES

- global variable: a variable defined outside all functions; it is accessible to all functions within its scope
- Easy way to share large amounts of data between functions
- Scope of a global variable is from its point of definition to when the program end
- Use sparingly

LOCAL VARIABLE LIFETIME

- A local variable only exists while its defining function is executing
- Local variables are destroyed when the function terminates
- Data cannot be retained in local variables between calls to the function in which they are defined

INITIALIZING LOCAL AND GLOBAL VARIABLES

- Local variables must be initialized by the programmer
- Global variables are initialized to **0** (numeric) or **`NULL`** (character) when the variable is defined

GLOBAL VARIABLES – WHY USE SPARINGLY?

Global variables make:

- Programs difficult to debug
- Functions that cannot easily be re-used in other programs
- Programs hard to understand

LOCAL AND GLOBAL VARIABLE NAMES

- Local variables can have same names as global variables
- When a function contains a local variable that has the same name as a global variable, the global variable is unavailable from within the function. The local definition "hides" or "shadows" the global definition.

STATIC LOCAL VARIABLES

- Local variables
  - Only exist while the function is executing
  - Are redefined each time function is called
  - Lose their contents when function terminates
- **`static`** local variables
  - Are defined with key word **`static`**

```
static int counter;
```

  - Are defined and initialized only the first time the function is executed
  - Retain their contents between function calls

GLOBAL VARIABLES AND GLOBAL CONSTANTS

- A global variable is any variable defined outside all the functions in a program.
- The scope of a global variable is the portion of the program from the variable definition to the end.
- This means that a global variable can be accessed by *all* functions that are defined after the global variable is defined.
- You should avoid using global variables because they make programs difficult to debug.
- Any global that you create should be *global constants*

Program 6-19

```
1  // This program calculates gross pay.
2  #include <iostream>
3  #include <iomanip>
4  using namespace std;
5
6  // Global constants
7  const double PAY_RATE = 22.55;     // Hourly pay rate
8  const double BASE_HOURS = 40.0;    // Max non-overtime hours
9  const double OT_MULTIPLIER = 1.5; // Overtime multiplier
10
11 // Function prototypes
12 double getBasePay(double);
13 double getOvertimePay(double);
14
15 int main()
16 {
17    double hours,              // Hours worked
18           basePay,            // Base pay
19           overtime = 0.0,     // Overtime pay
20           totalPay;           // Total pay
```

Global constants defined for values that do not change throughout the program's execution.

The constants are then used for those values throughout the program.

```
29      // Get overtime pay, if any.
30      if (hours > BASE_HOURS)
31         overtime = getOvertimePay(hours);
```

```
56      // Determine base pay.
57      if (hoursWorked > BASE_HOURS)
58         basePay = BASE_HOURS * PAY_RATE;
59      else
60         basePay = hoursWorked * PAY_RATE;
```

```
75      // Determine overtime pay.
76      if (hoursWorked > BASE_HOURS)
77      {
78         overtimePay = (hoursWorked - BASE_HOURS) *
79                  PAY_RATE * OT_MULTIPLIER;
```

DEFAULT ARGUMENTS

A default argument is an argument that is passed automatically to a parameter if the argument is missing on the function call.

- Must be a constant declared in prototype:

```
void evenOrOdd(int = 0);
```

- Can be declared in header if no prototype

- Multi-parameter functions may have default arguments for some or all of them:

```
int getSum(int, int=0, int=0);
```

Default arguments specified in the prototype

Program 6-24

```
1  // This program demonstrates default function arguments.
2  #include <iostream>
3  using namespace std;
4
5  // Function prototype with default arguments
6  void displayStars(int = 10, int = 1);
7
8  int main()
9  {
10    displayStars();      // Use default values for cols and rows.
11    cout << endl;
12    displayStars(5);     // Use default value for rows.
13    cout << endl;
14    displayStars(7, 3);  // Use 7 for cols and 3 for rows.
15    return 0;
16 }
```

(Program Continues)

DEFAULT ARGUMENTS

- If not all parameters to a function have default values, the defaultless ones are declared first in the parameter list:

```
int getSum(int, int=0, int=0);// OK
int getSum(int, int=0, int);  // NO
```

- When an argument is omitted from a function call, all arguments after it must also be omitted:

```
sum = getSum(num1, num2);     // OK
sum = getSum(num1, , num3);   // NO
```

ASK YOURSELF

- Can you describe the components in function definition and called function?
- Do you know how to call and define function?
- Do you know how to send data into a function?
- Do you know how to return a value from a function?